MACHINE LEARNING

UNIT-I
INTRODUCTION AND MACHINE LEARNING

Introduction:

Human Learning:

- Learning is typically referred to as the process of gaining information through observation.
- Why do we need to learn?
  - ✓ In our daily life, we need to carry out multiple activities.
  - ✓ It may be a task as simple as walking down the street or doing the homework. Or it may be some complex task like deciding the angle in which a rocket should be launched so that it can have a particular trajectory.
  - ✓ We keep learning more or in other words acquiring more information, the efficiency in doing the tasks keep improving
  - ✓ For example, with more knowledge, the ability to do homework with less number of mistakes increases. In the same way, information from past rocket launches helps in taking the right precautions and makes more successful rocket launch.
  - ✓ With more learning, tasks can be performed more efficiently.

Types of Human Learning:

Human learning happens in one of the three ways:

- Learning directly under expert guidance i.e., somebody who is an expert in the subject directly teaches us. **Ex.:** Parents, Teachers.
- Learning guided by knowledge gained from experts i.e., we build our own notion indirectly based on what we have learnt from the expert in the past. **Ex.:** Kids grouping objects based on colour, market campaign.
- Learning by self or self-learning i.e., we do it ourselves, may be after multiple attempts, some being unsuccessful. **Ex.** Baby learns walk through obstacles.

What is Machine Learning?

- Before answering the question 'What is machine learning?' more fundamental questions that peep into one's mind are
  - ✓ Do machines really learn?
  - ✓ If so, how do they learn?
  - ✓ Which problem can be considered as a well-posed learning problem?
  - ✓ What are the important features that are required to well-define a learning problem?

Machine learning is defined as

'A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.'

Past experience, it means past data related to the task. This data is an input to the machine from some source.

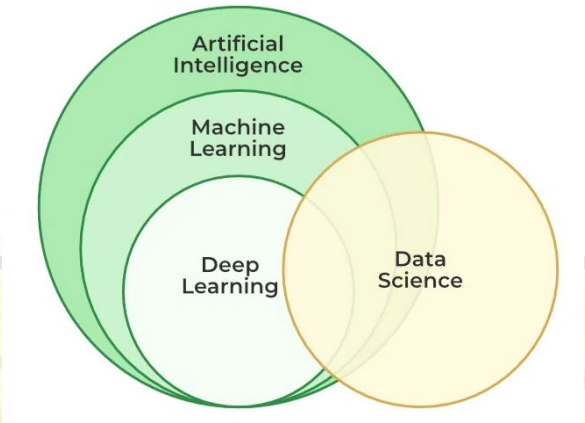
A machine can be considered to learn if it is able to gather experience by doing a certain task and improve its performance in doing the similar tasks in the future.

(or)

Machine Learning is defined as the study of computer algorithms for automatically constructing computer software through past experience and training data.

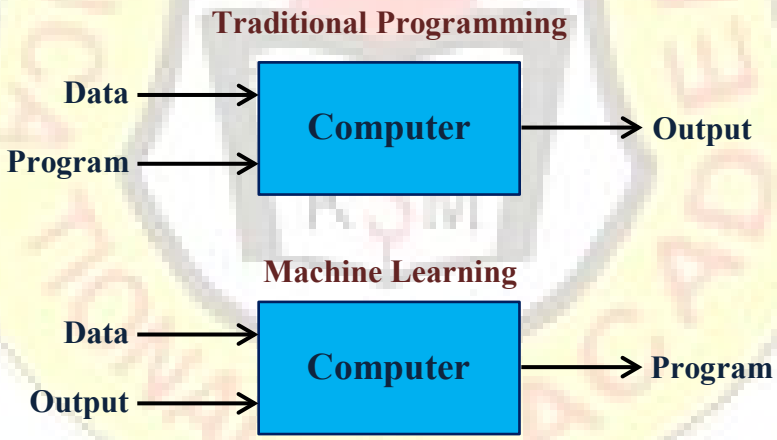
It is a branch of Artificial Intelligence and computer science that helps build a model based on

training data and make predictions and decisions without being constantly programmed. Machine Learning is used in various applications such as **email filtering, speech recognition, computer vision, self-driven cars, Amazon product recommendation,** etc.



Why Machine learning:

- Machine Learning is a branch of Artificial Intelligence that allows machines to learn and improve from experience automatically.
- It is defined as the field of study that gives computers the capability to learn without being explicitly programmed. It is quite different than traditional programming.



- Machine Learning is a subset of artificial intelligence(AI) that focus on learning from data to develop an algorithm that can be used to make a prediction.
- Machine Learning uses a data-driven approach; it is typically trained on historical data and then used to make predictions on new data.
- ML can find patterns and insights in large datasets that might be difficult for humans to discover.
- Machine Learning is the subset of AI. And now it is used in various AI-based tasks like Chatbot Question answering, self-driven car., etc.
- Machine learning is utilized for a variety of reasons across numerous fields due to its ability to extract patterns and insights from data without being explicitly programmed to do so. Some of the main reasons for employing machine learning include:
    - ✓ **Pattern Recognition:** Machine learning algorithms excel at identifying patterns within large datasets, which can be used for tasks such as image recognition, speech recognition, and natural language processing.
    - ✓ **Prediction and Forecasting:** Machine learning models can be trained to make

predictions and forecasts based on historical data, enabling businesses to anticipate future trends and make informed decisions.

- ✓ **Automation:** Machine learning algorithms can automate repetitive tasks and processes, leading to increased efficiency and cost savings. This is particularly valuable in industries such as manufacturing, finance, and healthcare.
- ✓ **Personalization:** Machine learning algorithms can analyze user behavior and preferences to provide personalized recommendations and experiences in areas such as e-commerce, entertainment, and social media.
- ✓ **Anomaly Detection:** Machine learning techniques can detect unusual patterns or outliers in data, which is valuable for fraud detection, cybersecurity, and predictive maintenance.
- ✓ **Optimization:** Machine learning algorithms can optimize complex systems and processes by analyzing data and identifying the most efficient strategies or configurations.
- ✓ **Insight Generation:** Machine learning can uncover hidden insights and correlations within data that may not be apparent through traditional analysis methods, leading to new discoveries and improved understanding in various fields.

- Overall, machine learning offers powerful tools for data analysis, prediction, automation, and decision-making, making it indispensable in today's data-driven world.

Examples of Machine Learning Problems

Learning algorithms have been successfully deployed in a variety of applications, including

- Text or document classification, e.g., spam detection;
- Natural language processing, e.g., morphological analysis, part-of-speech tagging, statistical parsing, named-entity recognition;
- Speech recognition, speech synthesis, speaker verification;
- Optical character recognition (OCR); Computational biology applications, e.g., protein function or structured prediction;
- Computer vision tasks, e.g., image recognition, face detection;
- Fraud detection (credit card, telephone) and network intrusion;
- Games, e.g., chess, backgammon;
- Unassisted vehicle control (robots, navigation);
- Medical diagnosis;
- Recommendation systems, search engines, information extraction systems.

This list is by no means comprehensive, and learning algorithms are applied to new applications every day. Moreover, such applications correspond to a wide variety of learning problems. Some major classes of learning problems are:

- ✓ **Classification:** Assign a category to each item. For example, document classification may assign items with categories such as politics, business, sports, or weather while image classification may assign items with categories such as landscape, portrait, or animal. The number of categories in such tasks is often relatively small, but can be large in some difficult tasks and even unbounded as in OCR, text classification, or speech recognition.
- ✓ **Regression:** Predict a real value for each item. Examples of regression include prediction of stock values or variations of economic variables. In this problem, the penalty for an incorrect prediction depends on the magnitude of the difference between the true and predicted values, in contrast with the classification problem, where there is typically no notion of closeness between various categories.

- ✓ **Ranking:** Order items according to some criterion. Web search, e.g., returning web pages relevant to a search query, is the canonical ranking example. Many other similar ranking problems arise in the context of the design of information extraction or natural language processing systems.
- ✓ **Clustering:** Partition items into homogeneous regions. Clustering is often performed to analyze very large data sets. For example, in the context of social network analysis, clustering algorithms attempt to identify "communities" within large groups of people.
- ✓ **Dimensionality reduction or manifold learning:** Transform an initial representation of items into a lower-dimensional representation of these items while preserving some properties of the initial representation. A common example involves preprocessing digital images in computer vision tasks.

The main practical objectives of machine learning consist of generating accurate predictions for unseen items and of designing efficient and robust algorithms to produce these predictions, even for large-scale problems.

To do so, a number of algorithmic and theoretical questions arise. Some fundamental questions include:

- ✓ Which concept families can actually be learned, and under what conditions?
- ✓ How well can these concepts be learned computationally?

Well-Posed Learning Problems

For example, a computer program that learns to play checkers might improve its performance as measured by its ability to win at the class of tasks involving playing checkers games, through experience obtained by playing games against itself. In general, to have a well-defined learning problem, we must identity these three features: the class of tasks, the measure of performance to be improved, and the source of experience.

A checkers learning problem:

- Task T: playing checkers
- Performance measure P: percent of games won against opponents
- Training experience E: playing practice games against itself

A handwriting recognition learning problem:

- Task T: recognizing and classifying handwritten words within images
- Performance measure P: percent of words correctly classified
- Training experience E: a database of handwritten words with given classifications

A robot driving learning problem:

- Task T: driving on public four-lane highways using vision sensors
- Performance measure P: average distance traveled before an error (as judged by human overseer)
- Training experience E: a sequence of images and steering commands recorded while observing a human driver

Structure of Learning:

Designing a Learning System

In order to illustrate some of the basic design issues and approaches to machine learning, let us consider designing a program to learn to play checkers.

A checkers learning problem:

- Task T: playing checkers
- Performance measure P: percent of games won against opponents

- Training experience E: playing practice games against itself

Choosing the Training Experience

The type of training experience available can have a significant impact on success or failure of the learner. One key attribute is whether the training experience provides direct or indirect feedback regarding the choices made by the performance system.

A second important attribute of the training experience is the degree to which the learner controls the sequence of training examples.

A third important attribute of the training experience is how well it represents the distribution of examples over which the final system performance P must be measured.

To proceed with our design, let us decide that our system will train by playing games against itself.

In order to complete the design of the learning system, we must choose

1. the exact type of knowledge to be, learned
2. a representation for this target knowledge
3. a learning mechanism

Choosing the Target Function

The next design choice is to determine exactly what type of knowledge will be learned and how this will be used by the performance program. We will find it useful to define one particular target function V among the many that produce optimal play.

The target value V(b) for an arbitrary board state b in B, as follows:

1. if b is a final board state that is won, then V(b) = 100
2. if b is a final board state that is lost, then V(b) = -100
3. if b is a final board state that is drawn, then V(b) = 0
4. if b is a not a final state in the game, then V(b) = $V(b^1)$, where b^1 is the best final board state that can be achieved starting from b and playing optimally until the end of the game (assuming the opponent plays optimally, as well).

In fact, we often expect learning algorithms to acquire only some approximation to the target function, and for this reason the process of learning the target function is often called function approximation (V^1).

For any given board state, the function V^1 will be calculated as a linear combination of the following board features:

- xl: the number of black pieces on the board
- x2: the number of red pieces on the board
- x3: the number of black kings on the board
- x4: the number of red kings on the board
- x5: the number of black pieces threatened by red (i.e., which can be captured on red's next turn)
- x6: the number of red pieces threatened by black

Thus, our learning program will represent $V^1(b)$ as a linear function of the form

$$\hat{V}(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

Where, w_o through w_6 are numerical coefficients, or weights, to be chosen by the learning algorithm

Partial design of a checkers learning program:

- Task T: playing checkers
- Performance measure P: percent of games won in the world tournament
- Training experience E: games played against itself
- Target function: $V: Board \rightarrow \Re$

- Target function representation

$$\hat{V}(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

The first three items above correspond to the specification of the learning task, whereas the final two items constitute design choices for the implementation of the learning program.

Choosing a Function Approximation Algorithm

In order to learn the target function V^1 we require a set of training examples, each describing a specific board state b and the training value $V_{train}(b)$ for b.

For instance, the following training example describes a board state b in which black has won the game (note $x_2 = 0$ indicates that red has no remaining pieces) and for which the target function value $V_{train}(b)$ is therefore +100.

$$\langle\langle x_1 = 3, x_2 = 0, x_3 = 1, x_4 = 0, x_5 = 0, x_6 = 0\rangle, +100\rangle$$

Estimating Training Values

Rule for estimating training values.

$$V_{train}(b) \leftarrow \hat{V}(Successor(b))$$

Adjusting the Weights

All that remains is to specify the learning algorithm for choosing the weights w_i to best fit the set of training examples $\{(b, V_{train}(b))\}$.)}. As a first step we must define what we mean by the best fit to the training data. One common approach is to define the best hypothesis, or set of weights, as that which minimizes the squared error E between the training values and the values predicted by the hypothesis V^1.

$$E \equiv \sum_{\langle b, V_{train}(b)\rangle \in \text{ training examples}} (V_{train}(b) - \hat{V}(b))^2$$

Thus, we seek the weights, or equivalently the V^1, that minimize E for the observed training examples.

Several algorithms are known for finding weights of a linear function that minimize E defined in this way. In our case, we require an algorithm that will incrementally refine the weights as new training examples become available and that will be robust to errors in these estimated training values. One such algorithm is called the least mean squares, or LMS training rule. For each observed training example it adjusts the weights a small amount in the direction that reduces the error on this training example.

LMS weight update rule.

For each training example $(b, V_{train}(b))$

- Use the current weights to calculate $V^1(b)$
- For each weight w_i, update it as

$$w_i \leftarrow w_i + \eta\,(V_{train}(b) - V^1(b))\,x_i$$

Here, η is a small constant (e.g., 0.1) that moderates the size of the weight update.

The Final Design

The final design of our checkers learning system can be naturally described by four distinct program modules that represent the central components in many learning systems. These four modules, summarized in Figure below, are as follows:

- The Performance System is the module that must solve the given performance task, in this case playing checkers, by using the learned target function(s). It takes an instance of a new problem (new game) as input and produces a trace of its solution (game history) as output. In our case, the strategy used by the Performance System to select its next move at each step is determined by the learned V^1 evaluation function. Therefore, we expect its performance to improve as this

evaluation function becomes increasingly accurate.

- The Critic takes as input the history or trace of the game and produces as output a set of training examples of the target function. As shown in the diagram, each training example in this case corresponds to some game state in the trace, along with an estimate V_{train}, of the target function value for this example. In our example, the Critic corresponds to the training rule.
- The Generalizer takes as input the training examples and produces an output hypothesis that is its estimate of the target function. It generalizes from the specific training examples, hypothesizing a general function that covers these examples and other cases beyond the training examples. In our example, the Generalizer corresponds to the LMS algorithm, and the output hypothesis is the function V^1 described by the learned weights $w_o, \ldots, w_6$.
- The Experiment Generator takes as input the current hypothesis (currently learned function) and outputs a new problem (i.e., initial board state) for the Performance System to explore. Its role is to pick new practice problems that will maximize the learning rate of the overall system. In our example, the Experiment Generator follows a very simple strategy: It always proposes the same initial game board to begin a new game. More sophisticated strategies could involve creating board positions designed to explore particular regions of the state space.

Many machine learning systems can-be usefully characterized in terms of these four generic modules.

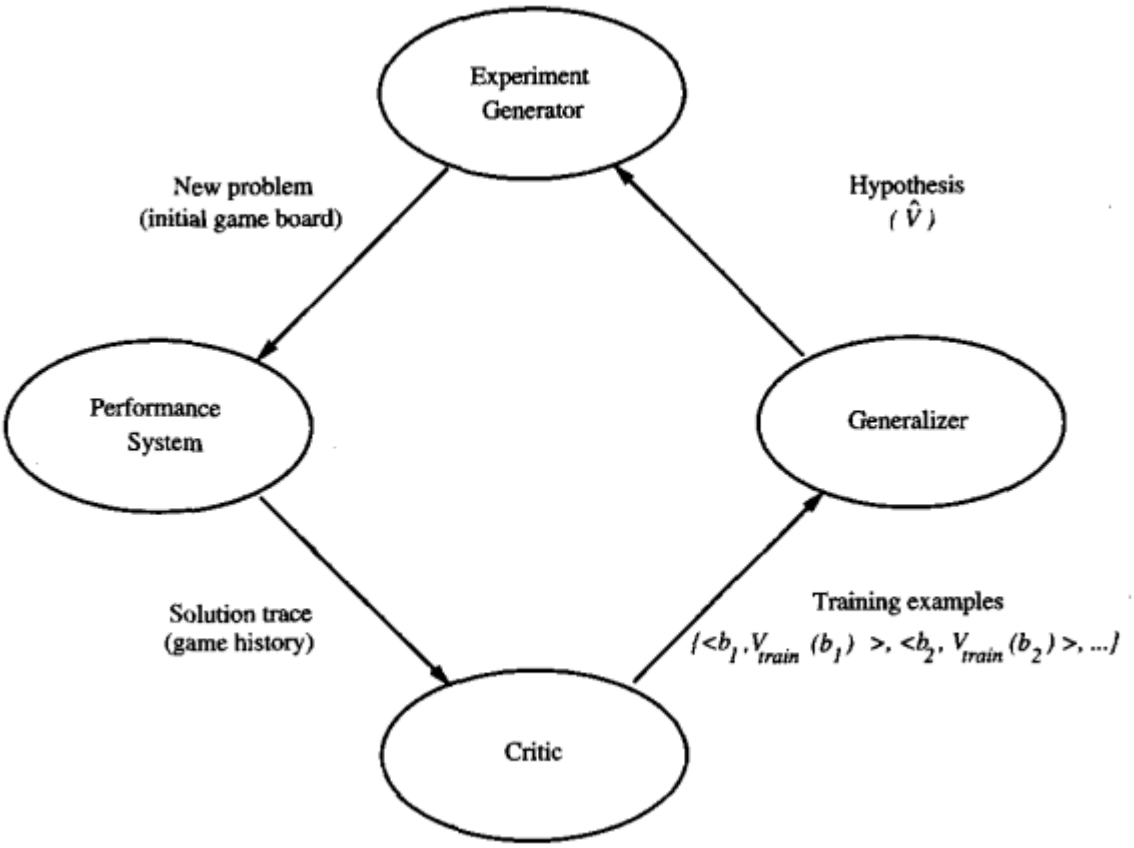


***Fig.*:-** Final design of the checkers learning program.

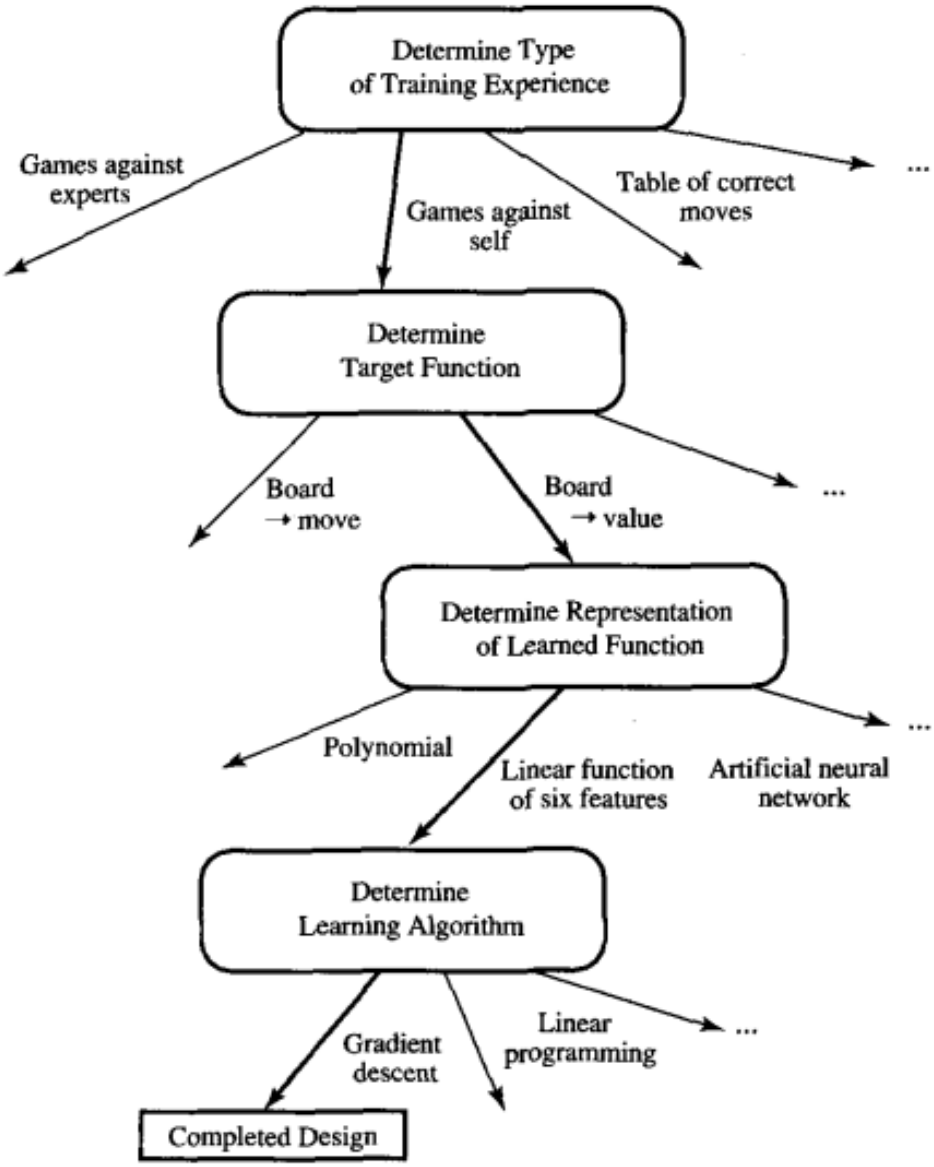
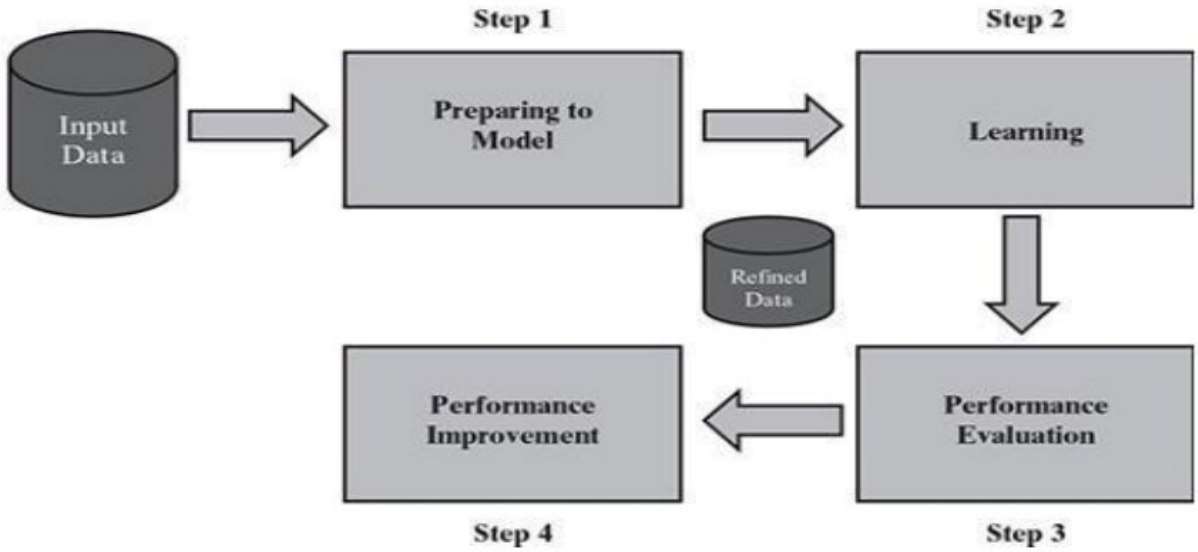


***Fig.*:-** Summary of choices in designing the checkers learning program.

Machine Learning Activities:

Machine Learning Process:



Activities Involved in Each Stage of Machine Learning

Step #	Step Name	Activities Involved
Step 1	Preparing to Model	• Understand the type of data in the given input data set • Explore the data to understand data quality • Explore the relationships amongst the data elements, e.g. inter-feature relationship • Find potential issues in data • Remediate data, if needed • Apply following pre-processing steps, as necessary: ✓ Dimensionality reduction ✓ Feature subset selection
Step 2	Learning	• Data partitioning/holdout • Model selection • Cross-validation
Step 3	Performance evaluation	• Examine the model performance, e.g. confusion matrix in case of classification • Visualize performance trade-offs using ROC curves
Step 4	Performance improvement	• Tuning the model • Ensembling • Bagging • Boosting

Learning versus Designing

- While learning is based on data, this other approach does not use data. It is a 'design' approach based on specifications, and is often discussed alongside the learning approach in pattern recognition literature.
- Machine learning is a powerful tool that drives everything from curated content recommendations to optimized user interfaces
- Machine learning answers questions about user behavior Machine learning customizes interfaces to users needs
- Digital product designers need to get familiar with machine learning
- Many warn that designers who don't start learning about ML will be left behind. But I haven't seen one that has explored what design and machine learning have to offer each other
- Design and machine learning function like a flywheel: when connected, each provides value to the other. Together, they open up new product experiences and business value
- Design helps machine learning gather better data
- Machine learning is a hungry beast. To deliver the best results, learning algorithms need vast amounts of detailed data, clean of any confounding factors or built-in biases
- Designers can help create user experiences that eliminate noise in data, leading to more accurate and efficient ML-powered applications
- Design helps set expectations and establish trust with users
- The main difference between the learning approach and the design approach is the role that data plays. In the design approach, the problem is well specified and one can analytically derive f without the need to see any data. In the learning approach, the problem is much less specified, and one needs data to pin down what f is.
- Both approaches may be viable in some applications, but only the learning approach is possible in many applications where the target function is unknown. We are not trying to compare the utility or the performance of the two approaches. We are just making the point that the design approach is distinct from learning.

Training versus Testing

In machine learning, training and testing are fundamental concepts that relate to the process of building and evaluating predictive models. Here's an overview of each:

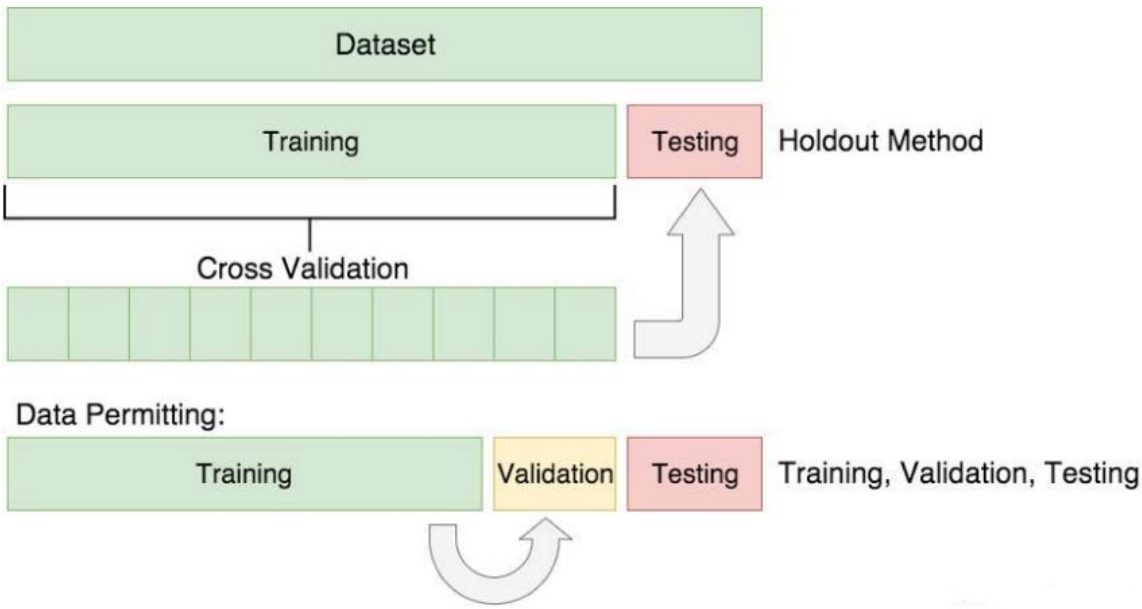
Training:

- Training refers to the phase where a machine learning algorithm learns patterns and relationships from the input data.
- During training, the algorithm is presented with a dataset containing input features and their corresponding target labels or outcomes (in the case of supervised learning).
- The algorithm adjusts its internal parameters or model structure based on this data to minimize the error or discrepancy between its predictions and the actual target values.
- The goal of training is to create a model that generalizes well to unseen data, meaning it can make accurate predictions on new, previously unseen examples.

Testing:

- Testing, also known as evaluation or validation, is the phase where the trained model is assessed for its performance on data it hasn't seen before.
- This unseen data is referred to as the testing dataset or validation dataset.
- The model's predictions on this dataset are compared with the actual target values to evaluate its performance metrics such as accuracy, precision, recall, F1-score, etc.
- Testing helps to estimate how well the model will generalize to new, unseen data and whether it has learned meaningful patterns from the training data or has just memorized it (overfitting).

It's important to note that the testing dataset should be separate from the training dataset to ensure an unbiased evaluation of the model's performance. Typically, the dataset is divided into training and testing sets, often using techniques like cross-validation to ensure robustness of the evaluation. Additionally, techniques like hyper-parameter tuning may be applied during training to optimize the model's performance on the testing dataset.



Characteristics of Machine learning tasks:

Machine learning is a subset of artificial intelligence that focuses on the development of algorithms and statistical models that enable computers to perform tasks without being explicitly programmed for those tasks. Some key characteristics of machine learning include:

- **Data-driven:** Machine learning algorithms rely on data to learn patterns and make predictions or decisions. The quality and quantity of data significantly influence the performance of the algorithm.
- **Algorithmic Learning:** Instead of being explicitly programmed, machine learning algorithms learn from data through iterative processes. They adjust their parameters to improve performance over time.
- **Generalization:** Machine learning models aim to generalize patterns learned from specific data to make predictions or decisions on new, unseen data. Generalization helps in making accurate predictions on new instances.
- **Model Evaluation:** Machine learning involves rigorous evaluation of models using various metrics such as accuracy, precision, recall, F1-score, etc., to assess their performance and determine their effectiveness in solving the given problem.
- **Flexibility and Scalability:** Machine learning algorithms can handle a wide range of tasks and data types, from structured to unstructured data. They can also scale to process large datasets efficiently.
- **Feature Extraction and Selection:** Machine learning often involves the process of feature extraction and selection, where relevant information from the data is identified and used to improve the performance of the model.
- **Supervised, Unsupervised, and Reinforcement Learning:** Machine learning can be categorized into different types based on the nature of the learning process. Supervised learning involves learning from labeled data, unsupervised learning involves learning from unlabeled data, and reinforcement learning involves learning from feedback received from the environment.
- **Iterative Improvement:** Machine learning models are often refined through iterative processes of training, evaluation, and fine-tuning. This iterative improvement helps in enhancing the model's performance over time.
- **Interpretability vs. Complexity:** Machine learning models vary in terms of their interpretability. Some models, like decision trees, are easy to interpret, while others, like deep neural networks, may be highly complex and less interpretable.
- **Application Diversity:** Machine learning finds applications across various domains such as healthcare, finance, marketing, gaming, recommendation systems, natural language processing, computer vision, and many more, highlighting its versatility and wide-ranging impact.

Predictive and descriptive tasks

	Predictive model	*Descriptive model*
Supervised learning	classification, regression	subgroup discovery
Unsupervised learning	predictive clustering	descriptive clustering, association rule discovery

***Fig*:-** An overview of different machine learning settings.

The rows refer to whether the training data is labelled with a target variable, while the columns indicate whether the models learned are used to predict a target variable or rather describe the given data.

Predictive Tasks:

Predictive tasks involve building models to predict future outcomes or classify new data points based on patterns learned from historical data.

- Predictive models try to predict certain value using the values in an input data set.
- The learning model attempts to establish a relation between the target feature, i.e. the feature being predicted, and the predictor features.
- These tasks are focused on making predictions or decisions. Some common predictive tasks include:
  - ✓ **Regression:** (value of target variable is real/continuous)Regression models are used to predict a continuous target variable based on one or more input features. For example, predicting house prices based on features like square footage, number of bedrooms, and location, Predicting revenue growth in successful year, Predicting rainfall amount in coming monsoon.
  - ✓ **Classification:** (Category/class of target variable is discrete) Classification models are used to assign labels or categories to input data points. This could involve binary classification (e.g., spam detection, fraud detection) or multi-class classification (e.g., win/loss/draw of a match, image recognition, sentiment analysis).
  - ✓ **Time Series Forecasting:** Time series forecasting involves predicting future values of a sequence of data points based on past observations. This is commonly used in finance, stock market prediction, weather forecasting, and demand forecasting.
  - ✓ **Recommendation Systems:** Recommendation systems predict the likelihood of a user's interest in a particular item or content. These systems are widely used in e-commerce, streaming platforms, and social media to personalize recommendations to users.
  - ✓ **Anomaly Detection:** Anomaly detection models identify rare events or outliers in data that deviate significantly from normal behavior. This is useful for fraud detection, network security, and quality control.

Descriptive Tasks:

Descriptive tasks involve exploring and summarizing the characteristics of a dataset without making predictions. These tasks are focused on gaining insights and understanding the structure of the data.

- Models for unsupervised learning or descriptive models are used to describe a data set or gain insight from a data set.
- There is no target feature or single feature of interest in case of unsupervised learning. Based on the value of all features, interesting patterns or insights are derived about the data set.
- Some common descriptive tasks include:
  - ✓ **Exploratory Data Analysis (EDA):** EDA involves visually exploring the dataset to understand its distribution, relationships between variables, and identify potential patterns or trends.
  - ✓ **Data Visualization:** Visualizing data through plots, charts, and graphs to convey insights and trends effectively. Visualization techniques help in understanding complex datasets and communicating findings to stakeholders.
  - ✓ **Clustering Analysis:** Clustering algorithms group similar data instances together based on their features without prior labels. This helps in identifying natural groupings or clusters within the data. (e.g., Customer grouping or segmentation based on social, demographic, ethnic, etc.; Grouping of music based on different aspects like genre, language, time period, etc. and Grouping of commodities in an inventory.)
  - ✓ **Dimensionality Reduction:** Dimensionality reduction techniques like Principal Component Analysis (PCA) and t-SNE reduce the number of features in the dataset while preserving essential information. This helps in visualizing high-dimensional data

and identifying important features.

- ✓ **Association Rule Mining:** Association rule mining identifies relationships or associations between different variables in the dataset. This is commonly used in market basket analysis to identify patterns in consumer behavior.

Machine learning Models:

Models: the output of machine learning

There is a considerable –not to say bewildering – range of machine learning models to choose from. One reason for this is the ubiquity of the tasks that machine learning aims to solve: classification, regression, clustering, and association discovery.

Models are classified into three groups, and are: geometric models, probabilistic models, and logical models.

These groupings are not meant to be mutually exclusive, and sometimes a particular kind of model has, for instance, both a geometric and a probabilistic interpretation.

Geometric Models:

- The instance space is the set of all possible or describable instances, whether they are present in our data set or not. Usually this set has some geometric structure.
- For instance, if all features are numerical, then we can use each feature as a coordinate in a Cartesian coordinate system.
- In Geometric models, features could be described as points in two dimensions (x- and y-axis) or a three-dimensional space (x, y, and z).
- Even when features are not intrinsically geometric, they could be modelled in a geometric manner (for example, temperature as a function of time can be modelled in two axes).
- One main advantage of geometric classifiers is that they are easy to visualize, as long as we keep to two or three dimensions.
- Note, high-dimensional spaces are hard to imagine but are nevertheless very common in machine learning.
- Geometric concepts that potentially apply to high-dimensional spaces are usually prefixed with 'hyper-': for instance, a decision boundary in an unspecified number of dimensions is called a ***hyperplane***.
- Geometric models are basically of two types :-
  1. Linear models
  2. Distance-based models

Linear models

- A geometric model that is constructed directly in instance space, using geometric concepts such as lines or planes are used to segment (classify) the instance space known as Linear Models.
- Linear models exist for all predictive tasks, including classification, probability estimation and regression. ***Example:*** Hours spent studying Vs. Marks scored by students, Amount of rainfall Vs. Agricultural yield, Electricity usage Vs. Electricity bill.
- There are different linear models like the least-squares method (a mathematical regression analysis), Support Vector Machine (which best segregates two or more classes using a hyper-plane).
- If there exists a linear decision boundary separating the two classes, we say that the data is ***linearly separable***.

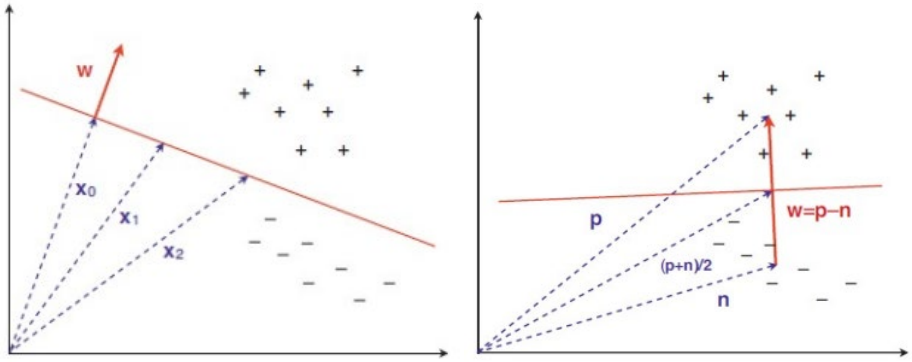


Fig.:- The basic linear classifier constructs a decision boundary by half-way intersecting the line between the positive and negative centres of mass. It is described by the equation $w \cdot x = t$, with $w = p - n$; the decision threshold can be found by noting that $(p + n)/2$ is on the decision boundary, and hence $t = (p - n) \cdot (p + n)/2 = (\|p\|^2 - \|n\|^2)/2$, where $\|x\|$ denotes the length of vector x.

- As we have seen, a linear decision boundary is defined by the equation $w \cdot x = t$, where w is a vector perpendicular to the decision boundary, x points to an arbitrary point on the decision boundary, and t is the decision threshold.
- One way to calculate these centres of mass is by averaging. For instance, if P is a set of n positive examples, then we can define

$$\mathbf{p} = \frac{1}{n}\sum_{x \in P} \mathbf{x}$$ and similarly for n.

- Because data is usually noisy, linear separability doesn't occur very often.
- One natural option is to prefer large margin classifiers, where the margin of a linear classifier is the distance between the decision boundary and the closest instance. ***Support vector machines***, are a powerful kind of linear classifier that find a decision boundary whose margin is as large as possible.

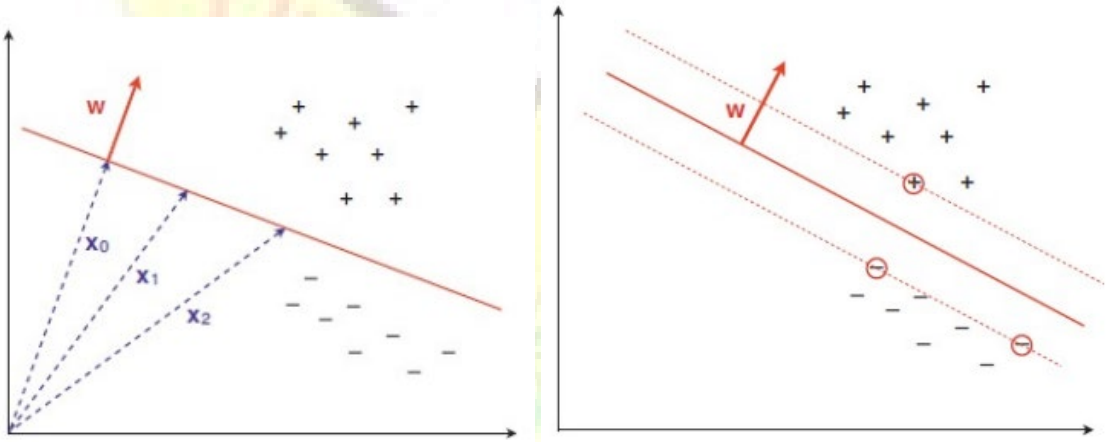


Fig.:- The decision boundary learned by a support vector machine from the linearly separable data from Figure (left). The decision boundary maximises the margin, which is indicated by the dotted lines. The circled data points are the support vectors.

- Geometric concepts, in particular linear transformations, can be very helpful to understand the similarities and differences between machine learning methods.
- Some algorithms are translation-invariant, i.e., insensitive to where we put the origin our coordinate system.
- Some algorithms may also be rotation-invariant, e.g., linear classifiers or support vector

machines; but many others aren't, including Bayesian classifiers. Similarly, some algorithms may be sensitive to non-uniform scaling.

- A very useful geometric concept in machine learning is the notion of distance.

Distance-based models

- In Distance-based models, we can use the geometric notion of distance to represent similarity.
- If the distance between two instances is small then the instances are similar in terms of their feature values, and so nearby instances would be expected to receive the same classification or belong to the same cluster.
- In a Cartesian coordinate system, distance can be measured by ***Euclidean distance***, which is the square root of the sum of the squared distances along each coordinate: $\sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$.
- A very simple distance-based classifier is ***nearest-neighbour classifier***. (Ex. k-nearest neighbor). KNN identifies the nearest points or groups for a query point. Common distance metrics include Euclidean distance, Manhattan distance, Minkowski Distance, Mahalanobis distance.
- For instance, instead of using Euclidean distance, which can be sensitive to outliers, other distances can be used such as ***Manhattan distance***, which sums the distances along each coordinate: $\sum_{i=1}^{d}|x_i - y_i|$.
- To classify a new instance, we retrieve from memory the most similar training instance (i.e., the training instance with smallest Euclidean distance from the instance to be classified), and simply assign that training instance's class.
- Some examples of Distance based models: k-nearest neighbor, k-means clustering, and hierarchical clustering.

Probabilistic Models

- Probabilistic models provides a framework for understanding the inherent uncertainty in data and making predictions based on probability distributions., like the Naïve Bayesian classifier.
- Unlike deterministic models that provide absolute values, probabilistic models make predictions based on probability distributions.
- These models are essential for learning patterns from data and making predictions on unseen examples. They find applications in various domains, including image and speech recognition, natural language processing, and recommendation systems.
- Therefore, Probabilistic models see features and target variables as random variables.
- The process of modelling represents and manipulates the level of uncertainty with respect to these variables.
- There are three types of probabilistic models:
    1. Generative models
    2. Discriminative models.
    3. Graphical models

1. Generative Models:-

- Generative models aim to model the joint distribution of input and output variables. They generate new data based on the probability distribution of the original dataset.
- Generative models are powerful because they can generate new data that resembles the training data.
- Generative models estimate the joint distribution P (Y, X). Once we know the joint distribution

for the generative models, we can derive any conditional or marginal distribution involving the same variables.

- Formally, the generative model estimates the conditional probability P(X|Y=y) for a given target y. For example, the Naive Bayes algorithm models P(x, y) and then transforms the probabilities into conditional probabilities P(y|x) by applying the Bayes rule.
- Some popular generative algorithms are: Naive Bayes Classifier, Gaussian Mixture Model, Hidden Markov Model.
- They can be used for tasks such as image and speech synthesis, language translation, and text generation.

2. Discriminative/Predictive models:-

- Discriminative models focus on modeling the conditional distribution of the output variable given the input variable.
- They learn decision boundaries that separate different classes of the output variable. These models are useful when we focus on making accurate predictions rather than generating new data.
- Discriminative/Predictive probability models use the idea of a conditional probability distribution P (Y |X) from which Y can be predicted from X.
- Unlike the generative algorithms, they don't model the underlying probability distributions.
- Some popular discriminative algorithms are: k-nearest neighbors (k-NN), Logistic regression, Support Vector Machines (SVMs), Decision Trees.
- They are used in tasks like image recognition, speech recognition, and sentiment analysis.

Generative vs. Discriminative Models

Parameters	Generative Algorithm	Discriminative Algorithm
Objective	Models the ***joint probability distribution*** of the input characteristics and labels in a generative manner (focuses on modeling).	Modeling the ***conditional probability*** distribution of labels given input attributes (focuses on a solution).
Estimates	P(x, y)	P(y | x)
Methodology	Creates new samples by learning the distribution of the underlying data.	Acquires the threshold of judgment that distinguishes various classes or categories.
Application	Text generation and image synthesis are examples of generative tasks.	Used in activities like sentiment analysis and image categorization.
Strength	Effective with inadequate or missing data and they has ability to express complex relationships between variables. Generative algorithms ***converge faster*** than discriminative algorithms. Hence, we prefer this model in case we have a small training dataset	Excellent at distinguishing between classes or categories is discrimination and generally perform better for classification tasks. The discriminative models converge to a ***smaller asymptotic error*** as we have large training dataset.
Weakness	May have trouble distinguishing different classes in large datasets.	Less useful when dealing with incomplete or missing data.

3. Graphical Models:-

- Graphical models represent conditional dependencies between variables using graphs.
- They are commonly used for tasks such as image recognition, natural language processing, and causal inference.

Naive Bayes Algorithm:

- The Naive Bayes algorithm is widely used in probabilistic models. It efficiently solves classification problems by leveraging the power of Bayes' theorem.
- The Naive Bayes Algorithm is used to calculate the probability of a given sample belonging to a particular class. This is done by calculating the posterior probability of each class given the sample and then selecting the class with the highest posterior probability as the predicted class.
- Despite its simplicity, the Naive Bayes algorithm has proven to be robust and effective in various real-world scenarios, from spam filtering to medical diagnosis.

Advantages of Probabilistic Models

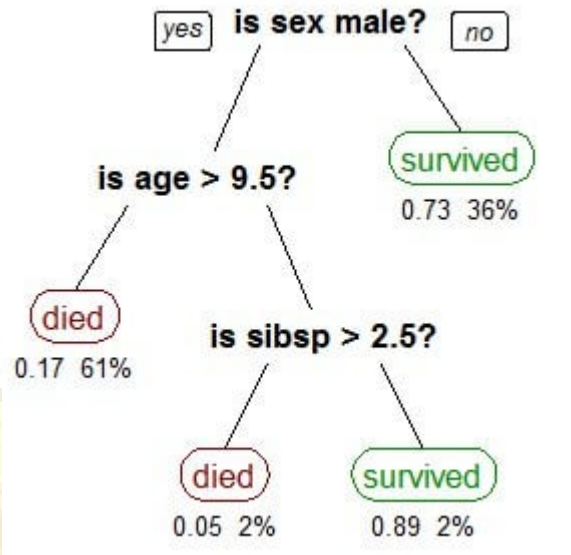
- Probabilistic models are an increasingly popular method in many fields, including artificial intelligence, finance, and healthcare.
- The main advantage of these models is their ability to take into account uncertainty and variability in data. This allows for more accurate predictions and decision-making, particularly in complex and unpredictable situations.
- Probabilistic models can also provide insights into how different factors influence outcomes and can help identify patterns and relationships within data.

Disadvantages of Probabilistic Models

- One of the disadvantages is the potential for overfitting, where the model is too specific to the training data and doesn't perform well on new data.
- Not all data fits well into a probabilistic framework, which can limit the usefulness of these models in certain applications.
- Another challenge is that probabilistic models can be computationally intensive and require significant resources to develop and implement.

Logical Models

- Logical models use a logical expression to divide the instance space into segments and hence construct grouping models. A logical expression is an expression that returns a Boolean value, i.e., a True or False outcome.
- "Logical" because models of this kind can easily be translated into rules that humans can understand, such as., if lottery = 1 then class = Y = spam. Such rules are easily arranged in a tree structure, which we refer to as a ***feature tree***.
- Feature trees whose leaves are labelled with classes are commonly called ***decision trees***.
- Logical models often take the form of decision trees, where features are used to iteratively partition the instance space.
- The leaves of the tree therefore correspond to rectangular areas in the instance space (or hyper-rectangles) and known as instance space segments.
- Depending on the task that we are solving, we can label the leaves with a class, a probability, a real value and so on.
- A simple example is the decision tree showing the survival of passengers on the Titanic:
  - ✓ If you were a female or a male younger than 9.5 years with strictly fewer than 3 siblings, your chances of survival were good.



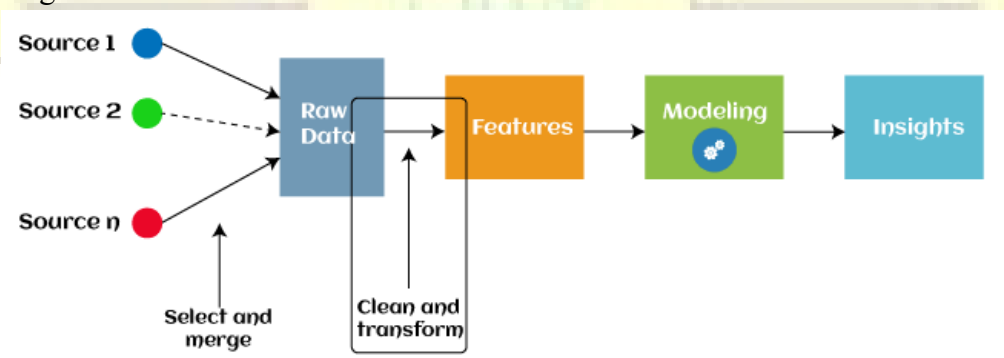
- Once the data is grouped using a logical expression, the data is divided into homogeneous groupings for the problem we are trying to solve.
- Logical models often have different, equivalent formulations.
- Tree-learning algorithms typically work in a top–down fashion are sometimes called a separate-and-conquer approach.
- An interesting aspect of logical models, which sets them aside from most geometric and probabilistic models, is that they can, to some extent, provide explanations for their predictions.
- There are two types of logical models: Tree based models and Rule based models.

Features:

- In machine learning, a feature (also known as a variable or attribute) is an individual measurable property or characteristic of a data point that is used as input for a machine learning algorithm.
- In machine learning, understanding and selecting appropriate features is crucial for effective algorithms in tasks like pattern recognition, classification, and regression.
- Features can be numerical, categorical, or text-based, and they represent different aspects of the data that are relevant to the problem.
  - ✓ For example, in a dataset of housing prices, features could include the number of bedrooms, the square footage, the location, and the age of the property. In a dataset of customer demographics, features could include age, gender, income level, and occupation.

What is Feature Engineering?

Feature engineering is the process of transforming raw data into features that are suitable for machine learning models.



In other words, it is the process of selecting, extracting, and transforming the most relevant

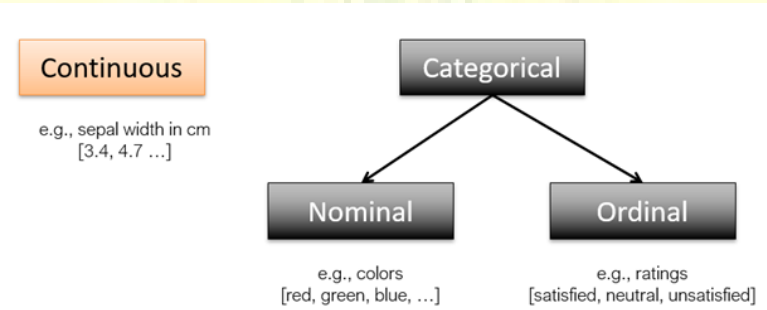
features from the available data to build more accurate and efficient machine learning models.

The success of machine learning models heavily depends on the quality of the features used to train them. Feature engineering involves a set of techniques that enable us to create new features by combining or transforming the existing ones. These techniques help to highlight the most important patterns and relationships in the data, which in turn helps the machine learning model to learn from the data more effectively.

- Feature engineering is an important pre-processing step for machine learning. It has three major elements: Feature Creation, Feature Transformation, Feature Construction and Feature Selection

Feature types:

1. **Numerical Features: (Quantitative or Continuous)**
   - Numerical features are continuous values that can be measured on a scale.
   - Examples of numerical features include age, height, weight, temperature and income.
   - These features can be directly used in machine learning algorithms.
   - For instance, when predicting someone's salary based on their age and education level, we use numerical features
   - Continuous features are often used in machine learning applications, since they can provide a more detailed representation of data than discrete or categorical features.
   - Continuous features can also be more useful than discrete features when it comes to optimizing models.
2. **Categorical Features: (Qualitative/Discrete: Nominal, Ordinal)**
   - Categorical features represent discrete categories or labels.
   - Categorical values that have a meaningful order, such as low, medium, or high, or beginner, intermediate, or advanced.
   - Boolean features: a special type of categorical feature that has only two possible values, such as true or false, or yes or no.
   - Examples include gender, color, country, product type or blood type.
   - These features are often encoded as integers or one-hot vectors before being used in machine learning models.
   - For instance, when classifying emails as "spam" or "not spam," the email sender's country could be a categorical feature.
   - Categorical features can be used to help predict what category something belongs to, based on other features. Categorical data can be thought of as a set of categories, and each category can be represented by a number.
   - Categorical features are of two types – nominal (Ex:- Blood group, Nationality and Gender) and ordinal. (Ex:- Grades, Ratings)



Features can be in the form of raw data that is very straightforward and can be derived from real-life as it is. However, not all problems can be solved using raw data or data in its original form. Ex: Color representation in RGB or HSV format.

3. **Structured features: (Text-specific)**
   - Structured or Text-specific features are essential for effectively representing and processing textual data.
   - All machine learning models need numerical data as input. So the text data in the data sets need to be transformed into numerical features.
   - Text data, or corpus which is the more popular keyword, is converted to a numerical representation following a process is known as vectorization.
   - Some common techniques for extracting features from text include:
     - ✓ ***Bag-of-Words (BoW) Model:*** The BoW model discards information about word order or contextual occurrence, focusing solely on word presence.
     - ✓ ***TF-IDF (Term Frequency-Inverse Document Frequency):*** It helps capture the uniqueness of words while considering their frequency.
     - ✓ ***Word Embeddings:*** Word embeddings capture semantic relationships between words and are useful for downstream tasks. Techniques like Word2Vec, GloVe, and FastText learn these embeddings from large text.

Feature Creation

Feature Creation is the process of generating new features based on domain knowledge or by observing patterns in the data. It is a form of feature engineering that can significantly improve the performance of a machine-learning model.

Types of Feature Creation:

- ***Domain-Specific:*** Creating new features based on domain knowledge, such as creating features based on business rules or industry standards.
- ***Data-Driven:*** Creating new features by observing patterns in the data, such as calculating aggregations or creating interaction features.
- ***Synthetic:*** Generating new features by combining existing features or synthesizing new data points.

Why Feature Creation?

- ***Improves Model Performance:*** By providing additional and more relevant information to the model, feature creation can increase the accuracy and precision of the model.
- **Increases Model Robustness:** By adding additional features, the model can become more robust to outliers and other anomalies.
- ***Improves Model Interpretability:*** By creating new features, it can be easier to understand the model's predictions.
- ***Increases Model Flexibility:*** By adding new features, the model can be made more flexible to handle different types of data.

Feature Transformation

It transforms the data – structured or unstructured, into a new set of features into a more suitable representation for the machine learning model. This is done to ensure that the model can effectively learn from the data.

Feature transformations aim at improving the utility of a feature by removing, changing or adding information.

We could order feature types by the amount of detail they convey: quantitative features are more detailed than ordinal ones, followed by categorical features, and finally Boolean features. The best-known feature transformations are those that turn a feature of one type into another of the next type

down this list. But there are also transformations that change the scale of quantitative features, or add a scale (or order) to ordinal, categorical and Boolean features.

There are two variants of feature transformation:

- Feature construction
- Feature extraction

Types of Feature Transformation:

- ***Normalization:*** Rescaling the features to have a similar range, such as between 0 and 1, to prevent some features from dominating others.
- ***Scaling:*** Scaling is a technique used to transform numerical variables to have a similar scale, so that they can be compared more easily. Rescaling the features to have a similar scale, such as having a standard deviation of 1, to make sure the model considers all features equally.
- ***Encoding:*** Transforming categorical features into a numerical representation. Examples are one-hot encoding and label encoding.
- ***Transformation:*** Transforming the features using mathematical operations to change the distribution or scale of the features. Examples are logarithmic, square root, and reciprocal transformations.

Why Feature Transformation?

- ***Improves Model Performance:*** By transforming the features into a more suitable representation, the model can learn more meaningful patterns in the data.
- ***Increases Model Robustness:*** Transforming the features can make the model more robust to outliers and other anomalies.
- ***Improves Computational Efficiency:*** The transformed features often require fewer computational resources.
- ***Improves Model Interpretability:*** By transforming the features, it can be easier to understand the model's predictions.

Feature Construction and Extraction:

Feature Construction and Extraction is the process of creating new features from existing ones to provide more relevant information to the machine learning model. This is done by transforming, combining, or aggregating existing features.

- It is a process discovers missing information about the relationships between features and augments the feature space by creating additional features.
- If there are 'n' features or dimensions in a data set, after feature construction 'm' more features or dimensions may get added. So at the end, the data set will become 'n + m' dimensional in case of feature construction.
- We can construct a new feature from two Boolean or categorical features by forming their Cartesian product.
- There are certain situations where feature construction is an essential activity before we can start with the machine learning task. For example:
  - ✓ when features have categorical value and machine learning needs numeric value inputs
  - ✓ when features having numeric (continuous) values and need to be converted to ordinal values
  - ✓ when text-specific feature construction needs to be done
- Some of the commonly used operators for combining the original features include
  - ✓ For Boolean features: Conjunctions, Disjunctions, Negation, etc.
  - ✓ For nominal features: Cartesian product, M of N, etc.

- ✓ For numerical features: Min, Max, Addition, Subtraction, Multiplication, Division, Average, Equivalence, Inequality, etc.

- The most popular feature extraction algorithms used in machine learning are PCA (Principal Component Analysis), SVD (Singular Value Decomposition) and LDA (Linear Discriminant Analysis)

Types of Feature Construction and Extraction:

- ***Dimensionality Reduction:*** Reducing the number of features by transforming the data into a lower-dimensional space while retaining important information. Examples are PCA (Principal Component Analysis) and t-SNE (T-distributed Stochastic Neighbor Embedding).
- ***Feature Combination:*** Combining two or more existing features to create a new one. For example, the interaction between two features.
- ***Feature Aggregation:*** Aggregating features to create a new one. For example, calculating the mean, sum, or count of a set of features.
- ***Feature Transformation:*** Transforming existing features into a new representation. For example, log transformation of a feature with a skewed distribution.

Why Feature Construction and Extraction?

- ***Improves Model Performance:*** By creating new and more relevant features, the model can learn more meaningful patterns in the data.
- ***Reduces Overfitting:*** By reducing the dimensionality of the data, the model is less likely to overfit the training data.
- ***Improves Computational Efficiency:*** The transformed features often require fewer computational resources.
- ***Improves Model Interpretability:*** By creating new features, it can be easier to understand the model's predictions.

Examples for Feature Transformation, Construction and Extraction:-

Encoding Categorical (Nominal) into Numerical:

Let's take the example of another data set on athletes. The data set has features age, city of origin, parents athlete (i.e. indicate whether any one of the parents was an athlete) and Chance of Win.

Age (Years)	City of origin	Parents athlete	Chance of win
18	City A	Yes	Y
20	City B	No	Y
23	City B	Yes	Y
19	City A	No	N
18	City C	Yes	N
22	City B	Yes	Y

Since the feature 'City of origin' has three unique values namely City A, City B, and City C, three dummy features namely origin_ city_A, origin_city_B, and origin_city_C is created and in the same way for Parents athlete and Chance of win. Then the dataset becomes

Age (Years)	origin_ city_A	origin_ city_B	origin_ city_C	parents_ athlete_Y	parents_ athlete_N	win_ chance_Y	win_ chance_N
18	1	0	0	1	0	1	0
20	0	1	0	0	1	1	0
23	0	1	0	1	0	1	0
19	1	0	0	0	1	0	1
18	0	0	1	1	0	0	1
22	0	1	0	1	0	1	0

We see that the features 'Parents athlete' and 'Chance of win' in the original data set can have two values only. So creating two features from them is a kind of duplication, since the value of one feature can be decided from the value of the other. To avoid this duplication, we can just leave one feature and eliminate the other. So, the dataset becomes

Age (Years)	origin_city_A	origin_city_B	origin_city_C	parents_athlete_Y	win_chance_Y
18	1	0	0	1	1
20	0	1	0	0	1
23	0	1	0	1	1
19	1	0	0	0	0
18	0	0	1	1	0
22	0	1	0	1	1

Encoding Categorical (Ordinal) into Numerical:

Let's take an example of a student data set. Let's assume that there are three variable – science marks, maths marks and grade.

marks_science	marks_maths	Grade
78	75	B
56	62	C
87	90	A
91	95	A
45	42	D
62	57	B

To transform the grade variable to a numeric variable, we can create a feature num_grade mapping a numeric value against each ordinal value. So, grades A, B, C, and D is mapped to values 1, 2, 3, and 4 in the transformed variable.

marks_science	marks_maths	num_grade
78	75	2
56	62	3
87	90	1
91	95	1
45	42	4
62	57	2

Transforming Numeric (Continuous) to Categorical:

Sometimes there is a need of transforming a continuous numerical variable into a categorical variable.

- For example, we may want to treat the real estate price prediction problem, which is a regression problem, as a real estate price category prediction, which is a classification problem.
- We can 'bin' the numerical data into multiple categories based on the data range.
- In the context of the real estate price prediction example, the original data set has a numerical feature apartment_price which can be transformed to a categorical variable price-grade.

apartment_ area	apartment_ price
4,720	23,60,000
2,430	12,15,000
4,368	21,84,000
3,969	19,84,500
6,142	30,71,000
7,912	39,56,000

→

apartment_ area	apartment_ grade
4,720	Medium
2,430	Low
4,368	Medium
3,969	Low
6,142	High
7,912	High

Text Specific Feature Construction:

The text is arguably the most predominant medium of communication. In social networks like Facebook or micro-blogging channels like Twitter or emails or short messaging services such as Whatsapp, text plays a major role in the flow of information.

- All machine learning models need numerical data as input. So the text data in the data sets need to be transformed into numerical features.
- Text data, or corpus which is the more popular keyword, is converted to a numerical representation following a process is known as vectorization.
- In this process, word occurrences in all documents belonging to the corpus are consolidated in the form of bag-of-words. There are three major steps that are followed:
    1. Tokenize
    2. Count
    3. Normalize
- Tokens are weighted with reducing importance when they occur in the majority of the documents.
- The number of occurrences of each token is counted, for each document.
- A matrix is then formed with each token representing a column and a specific document of the corpus representing each row.
- Each cell contains the count of occurrence of the token in a specific document. This matrix is known as a document-term matrix.

This	House	Build	Feeling	Well	Theatre	Movie	Good	Lonely	...
2	1	1	0	0	1	1	1	0	
0	0	0	1	1	0	0	0	0	
1	0	0	2	1	1	0	0	1	
0	0	0	0	1	0	1	1	0	
.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	

Feature Extraction Examples:

Principal Component Analysis:

- The objective of PCA is to make the transformation in such a way that
    1. The new features are distinct, i.e. the covariance between the new features, i.e. the principal components is 0.
    2. The principal components are generated in order of the variability in the data that it captures. Hence, the first principal component should capture the maximum variability, the second principal component should capture the next highest variability etc.
    3. The sum of variance of the new features or the principal components should be equal to the sum of variance of the original features.
- PCA works based on a process called eigenvalue decomposition of a covariance matrix of a data set. Below are the steps to be followed:
    1. First, calculate the covariance matrix of a data set.
    2. Then, calculate the eigenvalues of the covariance matrix.
    3. The eigenvector having highest eigenvalue represents the direction in which there is the highest variance. So this will help in identifying the first principal component.
    4. The eigenvector having the next highest eigenvalue represents the direction in which data has the highest remaining variance and also orthogonal to the first direction. So this helps in

identifying the second principal component.

5. Like this, identify the top 'k' eigenvectors having top 'k' eigenvalues so as to get the 'k' principal components.

Singular Value Decomposition:

- SVD of a matrix A (m × n) is a factorization of the form: $A = U \sum V$
- SVD is generally used in PCA, once the mean of each variable has been removed.
- Since it is not always advisable to remove the mean of a data attribute, especially when the data set is sparse (as in case of text data), SVD is a good choice for dimensionality reduction in those situations.
- SVD of a data matrix is expected to have the properties highlighted below:
  1. Patterns in the attributes are captured by the right-singular vectors, i.e. the columns of V.
  2. Patterns among the instances are captured by the left-singular, i.e. the columns of U.
  3. Larger a singular value, larger is the part of the matrix A that it accounts for and its associated vectors.
  4. New data matrix with 'k' attributes is obtained using the equation D = D × [v , v , … , v] Thus, the dimensionality gets reduced to k SVD is often used in the context of text data.

Linear Discriminant Analysis:

- The objective of LDA is similar to the sense that it intends to transform a data set into a lower dimensional feature space.
- Below are the steps to be followed:
  1. Calculate the mean vectors for the individual classes.
  2. Calculate intra-class and inter-class scatter matrices.
  3. Calculate eigenvalues and eigenvectors for S and S , where S is the intra-class scatter matrix and S is the inter-class scatter matrix

$$S_W = \sum_{i=1}^{c} S_i;$$

$$S_i = \sum_{x \in D_i}^{n} (x - m_i)(x - m_i)^T$$

where, m is the mean vector of the i-th class

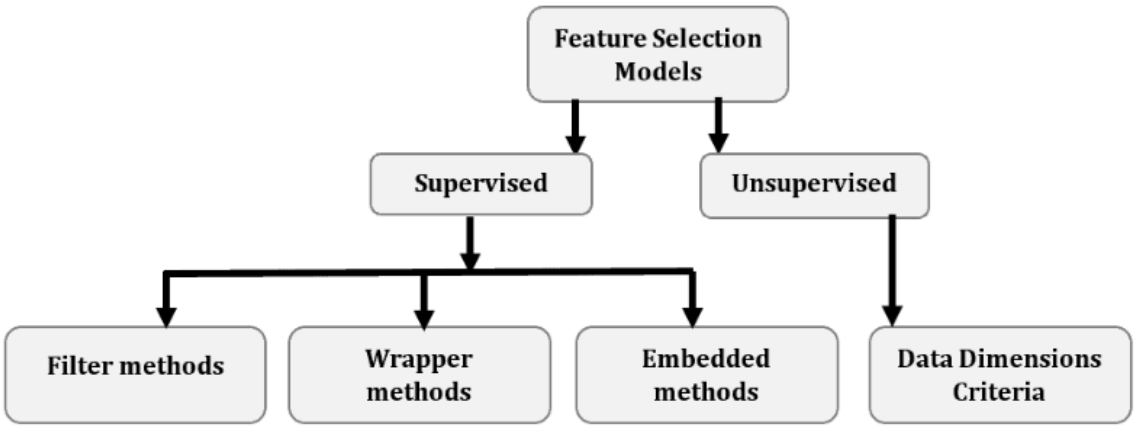
$$S_B = \sum_{i=1}^{c} N_i (m_i - m)(m_i - m)^T$$

where, mi is the sample mean for each class, m is the overall mean of the data set, Ni is the sample size of each class

4. Identify the top 'k' eigenvectors having top 'k' eigenvalues

Feature Selection: (Feature Subset Selection)

- Feature Selection is the process of selecting a subset of relevant features from the dataset to be used in a machine-learning model. It is an important step in the feature engineering process as it can have a significant impact on the model's performance.
- Not only will this speed up learning as fewer candidate features need to be considered, it also helps to guard against overfitting.
- Feature selection helps create more efficient, accurate, reduces dimensionality and interpretable machine learning models by focusing on relevant features while discarding noise and redundancy.



- Three main approaches for feature selection are:

1. **Filter Methods:**
   - ✓ These methods operate during pre-processing and select features independently of any specific machine learning algorithm.
   - ✓ They are computationally efficient and useful for removing duplicated, correlated (low correlatd), or redundant features.
   - ✓ Common filter methods include:
     1. ***Information Gain:*** Measures the reduction in entropy after splitting data based on a feature.
     2. ***Chi-Square Test:*** Assesses the independence between features and class labels.
     3. ***Fisher's Score:*** Evaluates feature relevance for classification tasks.

✓ Filter methods do not consider the learning algorithm but focus on feature characteristics

2. **Wrapper Methods:**
   - ✓ Wrapper methods evaluate feature subsets by training and testing the model using different subsets. We use a greedy strategy to form feature subsets.
   - ✓ They are computationally expensive but provide better feature subsets.
   - ✓ Common wrapper methods include:
     1. ***Forward Selection:*** Starts with an empty set of features and iteratively adds the most relevant feature.
     2. ***Backward Elimination:*** Begins with all features and removes the least relevant one in each iteration.
     3. ***Bi-Directional Elimination:*** Combines forward and backward approaches.
     4. ***Recursive Elimination:*** Recursively removes the least significant feature

3. **Embedded Methods:**
   - ✓ Embedded methods incorporate feature selection as part of the learning algorithm.
   - ✓ They are efficient and often used in regularized models.
   - ✓ Examples of embedded methods:
     1. ***Lasso Regression:*** Penalizes features with small coefficients, effectively performing feature selection.
     2. ***Random Forest Importance:*** Measures feature importance during tree construction.
     3. ***Gradient Boosting:*** Selects features based on their contribution to boosting performance

Pros and Cons:

Feature selection methods allow us to:

- ✓ reduce overfitting as less redundant data means less chance to make decisions based on noise;
- ✓ improve accuracy by removing misleading and unimportant data;
- ✓ reduce training time since data with fewer columns mean faster training.
- ✓ Decreases Computational Costs: A smaller number of features requires less computation and storage resources.
- ✓ Improves Interpretability: By reducing the number of features, it is easier to understand and interpret the results of the model.
- ➢ However, feature selection methods are hard to apply to high-dimensional data. The more features we have, the longer it takes for selection to complete.

Additional Topics to get Insights

Common issues in Machine Learning:

Although machine learning is being used in every industry and helps organizations make more informed and data-driven choices that are more effective than classical methodologies, it still has so many problems that cannot be ignored. Here are some common issues in Machine Learning that professionals face to inculcate ML skills and create an application from scratch.

1. Inadequate Training Data:

The major issue that comes while using machine learning algorithms is the lack of quality as well as quantity of data. Although data plays a vital role in the processing of machine learning algorithms, many data scientists claim that inadequate data, noisy data, and unclean data are extremely exhausting the machine learning algorithms. For example, a simple task requires thousands of sample data, and an advanced task such as speech or image recognition needs millions of sample data examples. Further, data quality is also important for the algorithms to work ideally, but the absence of data quality is also found in Machine Learning applications. Data quality can be affected by some factors as follows:

- ***Noisy Data*** **-** It is responsible for an inaccurate prediction that affects the decision as well as accuracy in classification tasks.
- ***Incorrect data*** **-** It is also responsible for faulty programming and results obtained in machine learning models. Hence, incorrect data may affect the accuracy of the results also.
- ***Generalizing of output data*** **-** Sometimes, it is also found that generalizing output data becomes complex, which results in comparatively poor future actions.

2. Poor quality of data:

As we have discussed above, data plays a significant role in machine learning, and it must be of good quality as well. Noisy data, incomplete data, inaccurate data, and unclean data lead to less accuracy in classification and low-quality results. Hence, data quality can also be considered as a major common problem while processing machine learning algorithms.

3. Non-representative training data:

To make sure our training model is generalized well or not, we have to ensure that sample training data must be representative of new cases that we need to generalize. The training data must cover all cases that are already occurred as well as occurring.

Further, if we are using non-representative training data in the model, it results in less accurate predictions. A machine learning model is said to be ideal if it predicts well for generalized cases and provides accurate decisions. If there is less training data, then there will be a sampling noise in the model, called the non-representative training set. It won't be accurate in predictions. To overcome this, it will be biased against one class or a group.

Hence, we should use representative data in training to protect against being biased and make accurate predictions without any drift.

4. Overfitting and Underfitting:

Overfitting:

Overfitting is one of the most common issues faced by Machine Learning engineers and data scientists. Whenever a machine learning model is trained with a huge amount of data, it starts capturing noise and inaccurate data into the training data set. It negatively affects the performance of the model. Let's understand with a simple example where we have a few training data sets such as 1000 mangoes, 1000 apples, 1000 bananas, and 5000 papayas. Then there is a considerable probability of identification of an apple as papaya because we have a massive amount of biased data in the training data set; hence prediction got negatively affected. The main reason behind overfitting is using non-linear methods used in machine learning algorithms as they build non-realistic data models. We can overcome overfitting by using linear and parametric algorithms in the machine learning models.

Methods to reduce overfitting:

- Increase training data in a dataset.
- Reduce model complexity by simplifying the model by selecting one with fewer parameters
- Ridge Regularization and Lasso Regularization
- Early stopping during the training phase
- Reduce the noise
- Reduce the number of attributes in training data.
- Constraining the model.

Underfitting:

Underfitting is just the opposite of overfitting. Whenever a machine learning model is trained with fewer amounts of data, and as a result, it provides incomplete and inaccurate data and destroys the accuracy of the machine learning model.

Underfitting occurs when our model is too simple to understand the base structure of the data, just like an undersized pant. This generally happens when we have limited data into the data set, and we try to build a linear model with non-linear data. In such scenarios, the complexity of the model destroys, and rules of the machine learning model become too easy to be applied on this data set, and the model starts doing wrong predictions as well.

Methods to reduce Underfitting:

- Increase model complexity
- Remove noise from the data
- Trained on increased and better features
- Reduce the constraints
- Increase the number of epochs to get better results.

5. Monitoring and maintenance:

As we know that generalized output data is mandatory for any machine learning model; hence, regular monitoring and maintenance become compulsory for the same. Different results for different actions require data change; hence editing of codes as well as resources for monitoring them also become necessary.

6. Getting bad recommendations:

A machine learning model operates under a specific context which results in bad recommendations and concept drift in the model. Let's understand with an example where at a specific time customer is looking for some gadgets, but now customer requirement changed over time but still machine learning model showing same recommendations to the customer while customer expectation

has been changed. This incident is called a Data Drift. It generally occurs when new data is introduced or interpretation of data changes. However, we can overcome this by regularly updating and monitoring data according to the expectations.

7. Lack of skilled resources:

Although Machine Learning and Artificial Intelligence are continuously growing in the market, still these industries are fresher in comparison to others. The absence of skilled resources in the form of manpower is also an issue. Hence, we need manpower having in-depth knowledge of mathematics, science, and technologies for developing and managing scientific substances for machine learning.

8. Customer Segmentation:

Customer segmentation is also an important issue while developing a machine learning algorithm. To identify the customers who paid for the recommendations shown by the model and who don't even check them. Hence, an algorithm is necessary to recognize the customer behavior and trigger a relevant recommendation for the user based on past experience.

9. Process Complexity of Machine Learning:

The machine learning process is very complex, which is also another major issue faced by machine learning engineers and data scientists. However, Machine Learning and Artificial Intelligence are very new technologies but are still in an experimental phase and continuously being changing over time. There is the majority of hits and trial experiments; hence the probability of error is higher than expected. Further, it also includes analyzing the data, removing data bias, training data, applying complex mathematical calculations, etc., making the procedure more complicated and quite tedious.

10. Data Bias:

Data Biasing is also found a big challenge in Machine Learning. These errors exist when certain elements of the dataset are heavily weighted or need more importance than others. Biased data leads to inaccurate results, skewed outcomes, and other analytical errors. However, we can resolve this error by determining where data is actually biased in the dataset. Further, take necessary steps to reduce it.

Methods to remove Data Bias:

- Research more for customer segmentation.
- Be aware of your general use cases and potential outliers.
- Combine inputs from multiple sources to ensure data diversity.
- Include bias testing in the development process.
- Analyze data regularly and keep tracking errors to resolve them easily.
- Review the collected and annotated data.
- Use multi-pass annotation such as sentiment analysis, content moderation, and intent recognition.

11. Lack of Explainability

This basically means the outputs cannot be easily comprehended as it is programmed in specific ways to deliver for certain conditions. Hence, a lack of explainability is also found in machine learning algorithms which reduce the credibility of the algorithms.

12. Slow implementations and results

This issue is also very commonly seen in machine learning models. However, machine learning models are highly efficient in producing accurate results but are time-consuming. Slow programming, excessive requirements' and overloaded data take more time to provide accurate results than expected. This needs continuous maintenance and monitoring of the model for delivering accurate results.

13. Irrelevant features

Although machine learning models are intended to give the best possible outcome, if we feed garbage data as input, then the result will also be garbage. Hence, we should use relevant features in our

training sample. A machine learning model is said to be good if training data has a good set of features or less to no irrelevant features.

Types of machine learning:

Supervised learning:

In supervised learning, the system is taught with the aid of using examples. The operator offers the system learning set of rules with a dataset that consists of preferred inputs and outputs, and the set of rules ought to locate a technique to decide a way to arrive at the inputs and outputs of the one.

While the operator is aware of the appropriate solutions to the problem, the set of rules identifies styles in data, learns from observations, and makes predictions.

The set of rules makes predictions and is corrected by the operator – and this method maintains till the set of rules achieves an excessive stage of accuracy/performance.

Unsupervised learning:

Here, the machine learning set of rules studies facts to discover patterns. There isn't any solution key or human operator to offer instruction.

Instead, the device determines the correlations and relationships through analysing information. In an unsupervised learning process, the device studying a set of rules is left to interpret massive information units and deal with that information accordingly.

The set of rules attempts to organize those facts in some way to explain its structure. This may suggest grouping the information into clusters or arranging it in a manner that appears greater organized.

Semi-supervised learning:

Semi-supervised learning is much like supervised learning; however as an alternative use each labelled and unlabelled record.

Labelled records are basically records that have significant tags in order that the set of rules can apprehend the records, while unlabelled records lack those records. By the use of this combination, device learning algorithms can learn how to label unlabelled records.

Reinforcement learning:

Reinforcement learning specializes in regimented learning processes, in which a system learning set of rules is supplied with a fixed set of actions, parameters, and cease values.

By defining the rules, the system learning set of rules then attempts to discover one-of-a-kind alternatives and possibilities, tracking and comparing every end result to decide which one is optimal. Reinforcement learning teaches the system trial and error. It learns from past reports and starts to conform its technique in reaction to the scenario to obtain the first-rate viable end result.